Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

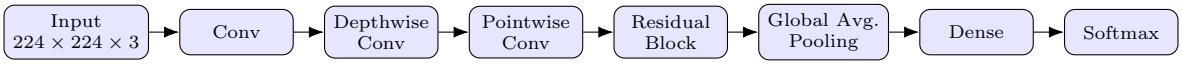
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization strategy can improve convergence and reduce problems such as vanishing or exploding activations.

The following four initialization strategies were investigated:

- Zero Initialization
- Random Initialization
- Xavier/Glorot Initialization
- He Initialization

The initialization methods were compared using training loss and validation accuracy over the training epochs.

Plot 1: Training Loss vs. Epoch

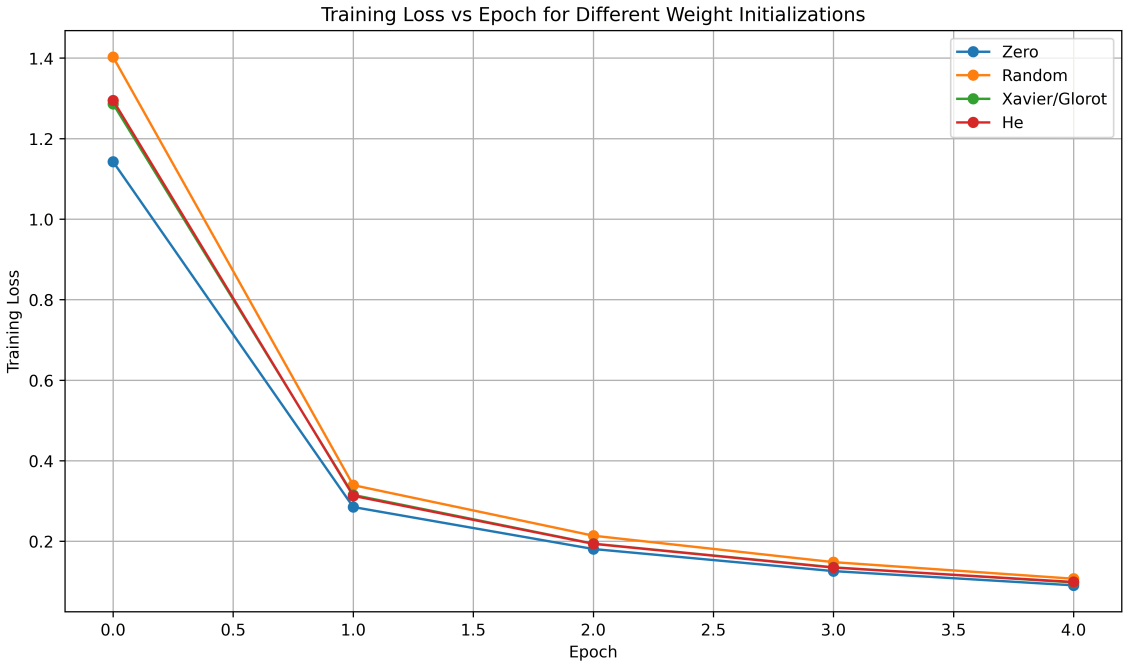


Figure 1: Training Loss for Different Weight Initialization Strategies

Inference: The plot compares the training-loss convergence of the four weight initialization strategies. The initialization methods show different convergence behaviour, with He initialization providing a stable reduction in training loss. This is suitable for networks using ReLU-based activations because it maintains an appropriate variance of activations during training.

Plot 2: Validation Accuracy vs. Epoch

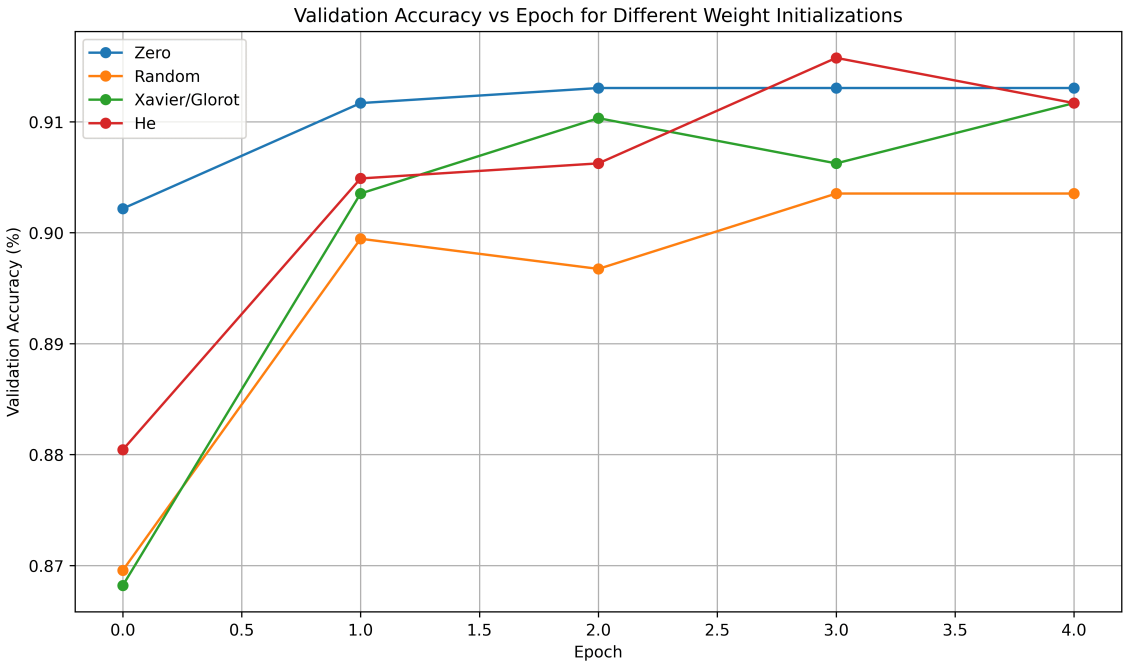


Figure 2: Validation Accuracy for Different Weight Initialization Strategies

Inference: The validation-accuracy curves show the effect of initialization on model convergence and generalization. Among the tested strategies, He initialization achieved the best validation performance and exhibited stable convergence compared with the other initialization methods.

Comparison of Initialization Strategies

Initialization	Final Val. Accuracy	Best Val. Accuracy	Training Time (s)
Zero	91.30%	91.30%	41.96
Random	90.35%	90.35%	41.71
Xavier/Glorot	91.17%	91.17%	41.40
He	91.17%	91.58%	41.91

Inference: The experiments demonstrate that weight initialization has a significant effect on optimization and generalization. He initialization was selected as the best-performing initialization strategy for the subsequent experiments.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. To study the effect of regularization on model generalization, four configurations were compared:

- No Regularization
- L2 Regularization
- Dropout
- Batch Normalization

Training and validation accuracy as well as training and validation loss were monitored throughout the training process.

Plot 3: Training and Validation Accuracy vs. Epoch

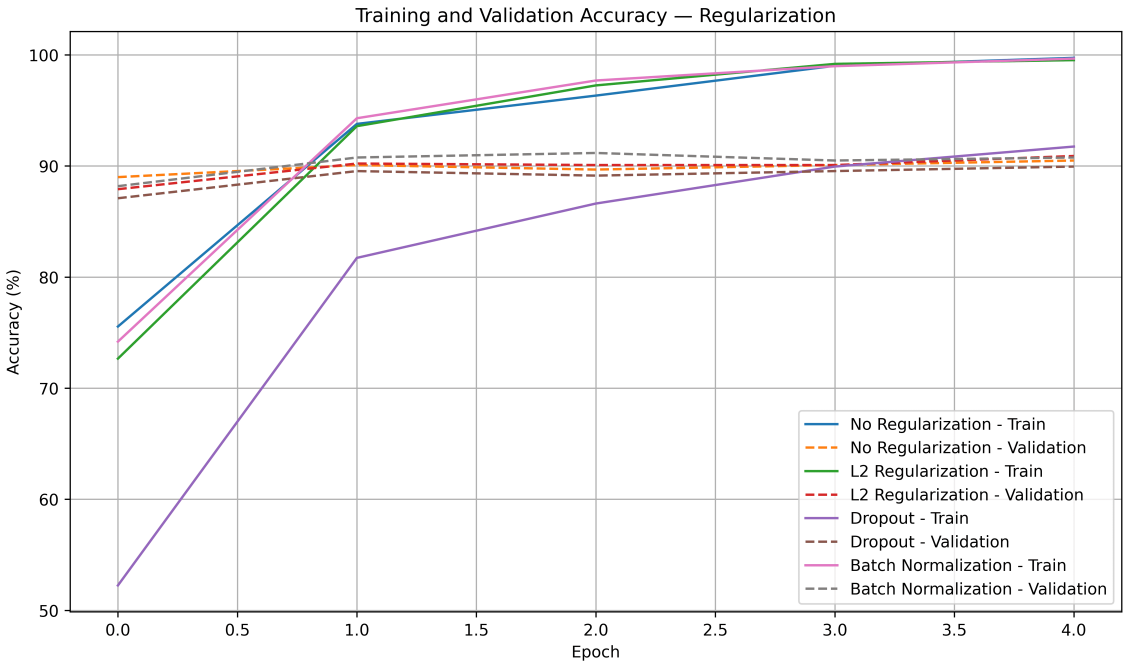


Figure 3: Training and Validation Accuracy for Different Regularization Methods

Inference: The plot shows the training and validation accuracy for the different regularization strategies. The difference between training and validation accuracy indicates the generalization gap. Regularization helps reduce excessive dependence on the training data and can therefore improve the model's ability to generalize.

Plot 4: Training and Validation Loss vs. Epoch

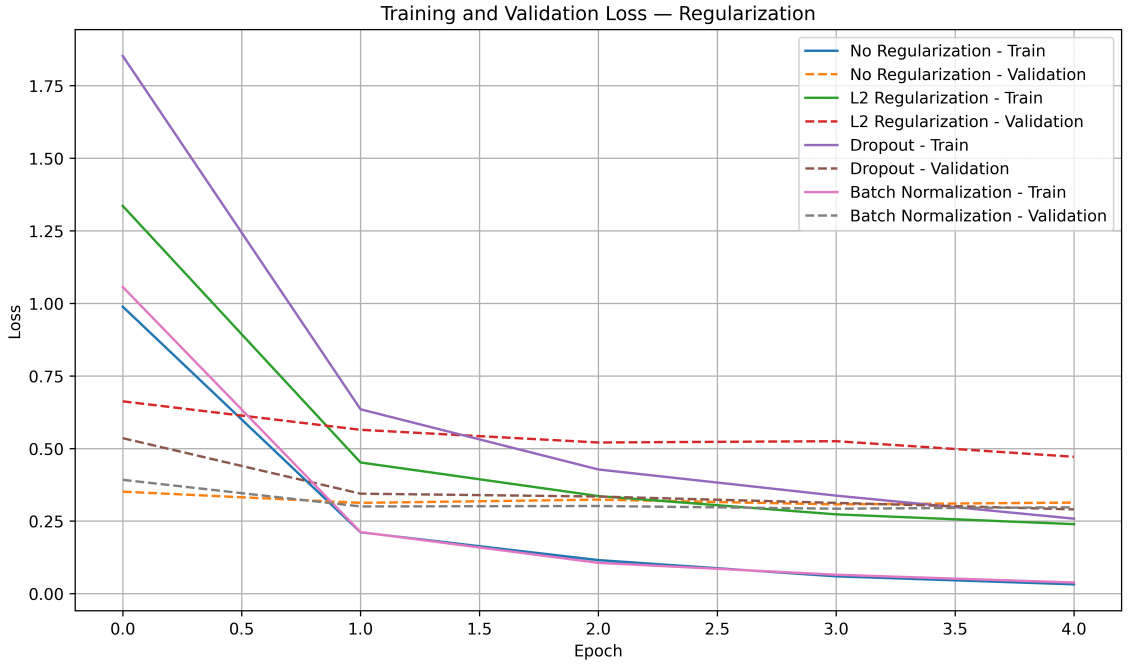


Figure 4: Training and Validation Loss for Different Regularization Methods

Inference: The training and validation loss curves illustrate the presence of overfitting and the effect of regularization. A model is likely to overfit when training loss continues to decrease while validation loss starts increasing. Regularization reduces this gap by constraining the model and improving generalization.

Comparison of Regularization Methods

Method	Purpose	Observed Effect
No Regularization	Provides a baseline without an explicit regularization technique	Used as the reference configuration for identifying overfitting.
L2 Regularization	Penalizes large model weights	Helps constrain the model and reduce overfitting.
Dropout	Randomly disables neurons during training	Reduces co-adaptation and improves generalization.
Batch Normalization	Normalizes activations during training	Improves training stability and was the best-performing regularization configuration in the experiment.

Inference: The comparison shows that the choice of regularization affects both convergence and generalization. Among the tested approaches, Batch Normalization produced the best overall validation performance and was therefore selected as the best regularization strategy for subsequent experiments.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training. It helps stabilize the learning process by maintaining activations within a suitable range.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i}{\sqrt{\sigma_B^2 + \epsilon}}.$$

More precisely, the normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider the mini-batch

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2+4+6+8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these values become the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable parameters γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

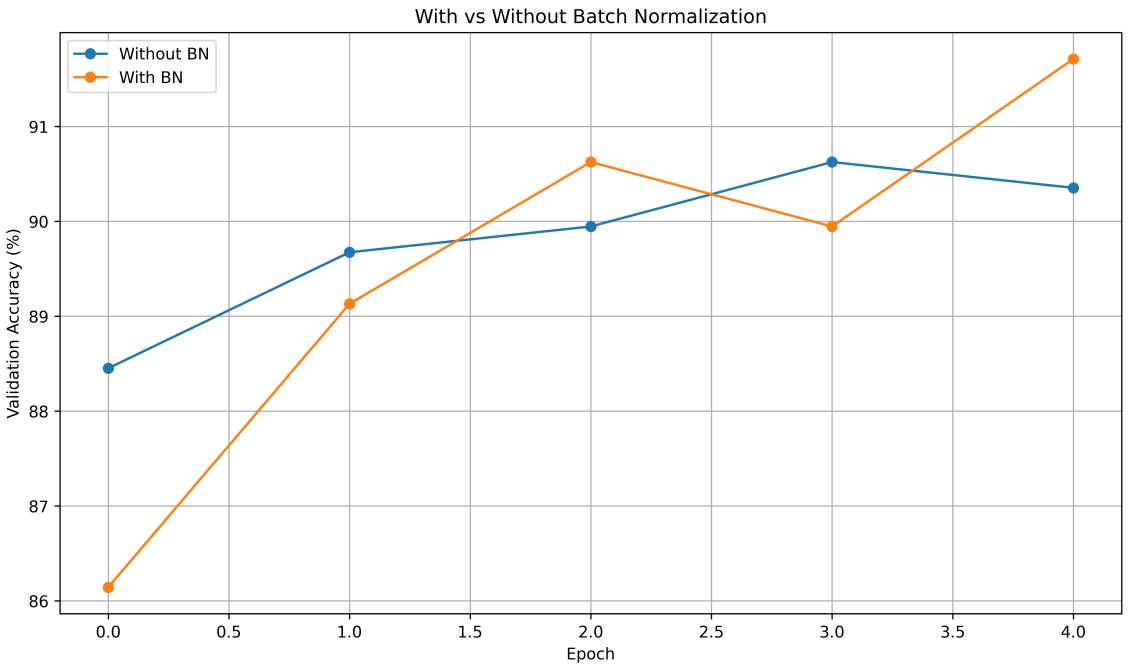


Figure 5: Validation Accuracy With and Without Batch Normalization

Inference: The plot compares validation accuracy with and without Batch Normalization. The Batch Normalization configuration shows improved and more stable validation performance, supporting its use as an effective training and regularization technique in the experiment.

8. Optimization Algorithms

Optimization algorithms determine how the model weights are updated during training. Different optimizers can lead to different convergence speeds, stability and final validation performance.

The following optimization algorithms were compared:

- Stochastic Gradient Descent (SGD)
- Momentum
- RMSProp
- Adam

All four optimizers were evaluated using the same CNN architecture and experimental conditions so that their effect on training behaviour could be compared.

Plot 6: Training Loss vs. Epoch for Different Optimizers

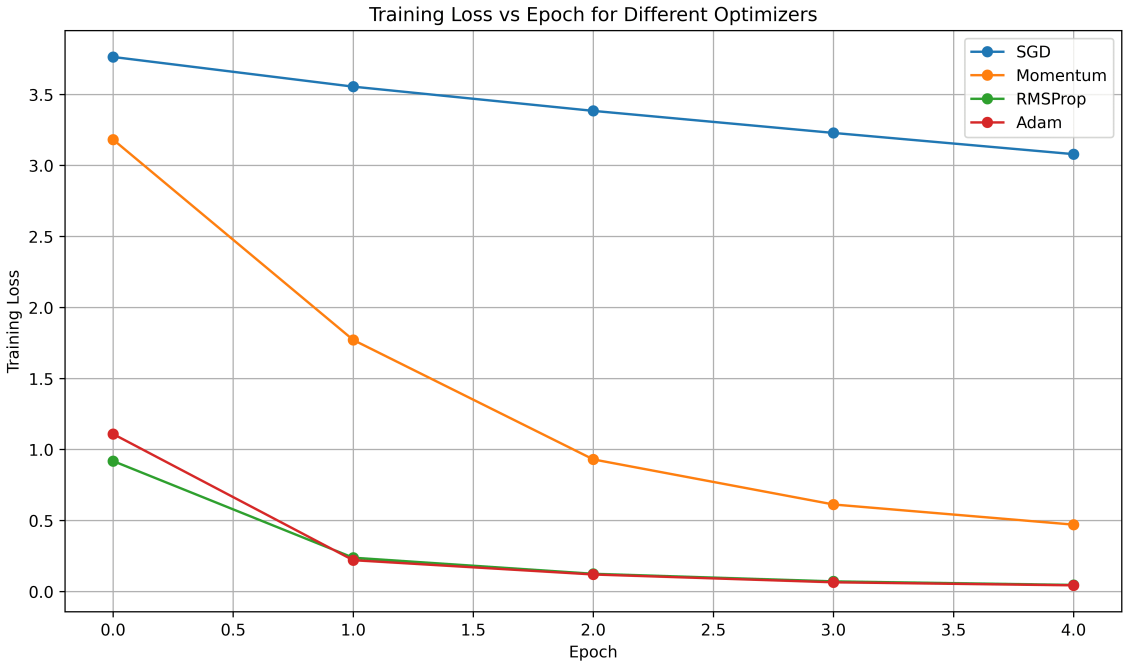


Figure 6: Training Loss for Different Optimization Algorithms

Inference: The training-loss curves show how quickly the different optimization algorithms minimize the loss during training. Adaptive optimizers generally provide faster convergence, while the shape and smoothness of the curves indicate the stability of the optimization process.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

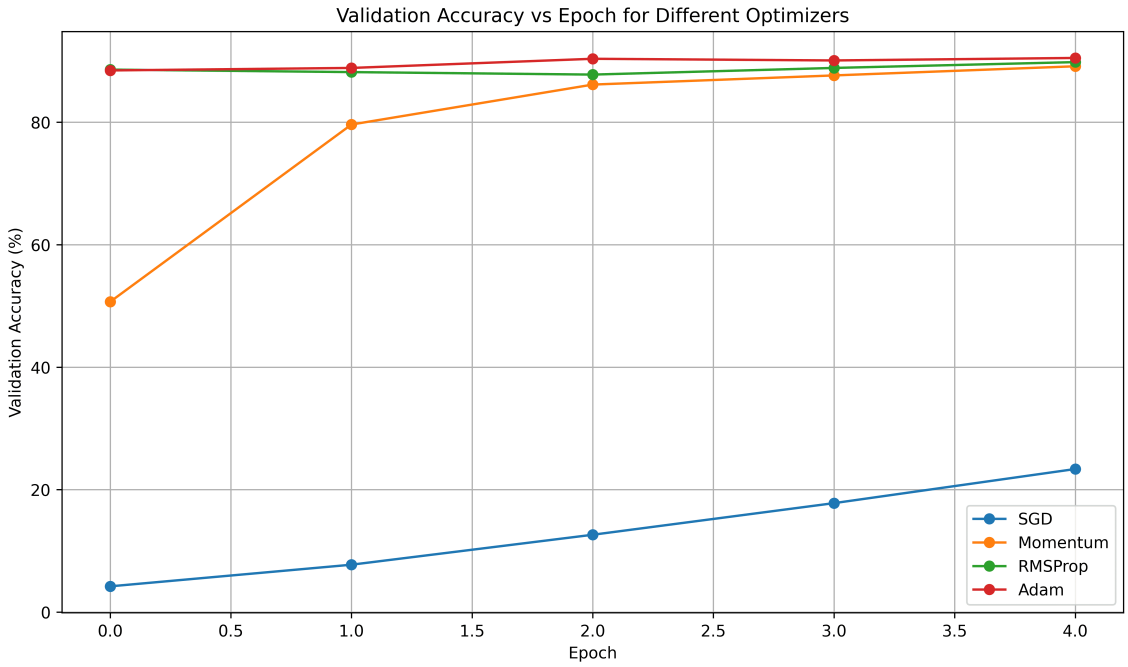


Figure 7: Validation Accuracy for Different Optimization Algorithms

Inference: The validation-accuracy curves demonstrate the difference in generalization performance among the optimizers. The optimizer that reaches high validation accuracy quickly and maintains stable performance provides more effective convergence for this classification task.

Optimizer Comparison

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	3.079395	23.369566%	5	42.872424
Momentum	0.470831	89.130437%	5	42.319794
RMSProp	0.046401	89.807981%	5	42.349535
Adam	0.043456	90.489131%	5	42.808264

Inference: Adam achieved the highest validation accuracy of 90.49% and the lowest final loss of 0.043456 among the four optimizers. RMSProp performed closely behind with a validation accuracy of 89.81% and a final loss of 0.046401.

SGD performed significantly worse, achieving only 23.37% validation accuracy with a final loss of 3.079395. The training times were similar for all optimizers, indicating that the main difference in this experiment was optimization effectiveness rather than computational time.

9. CNN Hyperparameter Tuning

Hyperparameters control the training behaviour and architecture of a CNN. In this experiment, important training hyperparameters were investigated to understand their effect on validation performance.

The following hyperparameters were considered:

Hyperparameter	Values Investigated
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

During the controlled hyperparameter experiments, one hyperparameter was changed at a time while keeping the remaining settings fixed. This allows the effect of each hyperparameter to be studied independently.

CNN Concepts

Kernel/Filter: A kernel is a small learnable matrix that slides over the input image to extract features such as edges, textures and patterns.

Stride: Stride specifies the number of pixels by which the kernel moves across the input.

Padding: Padding adds additional pixels around the input boundary and can be used to preserve spatial dimensions.

Pooling: Pooling reduces the spatial dimensions of feature maps while retaining important information.

Number of Channels: Channels represent the depth of an input or feature map. An RGB image, for example, has three input channels.

Depthwise Separable Convolution: This operation separates spatial filtering from channel-wise feature combination, reducing the number of computations compared with a standard convolution.

For a convolution operation, the output dimension is calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is the input size, K is the kernel size, P is the padding and S is the stride.

Plot 8: Learning Rate vs. Validation Accuracy

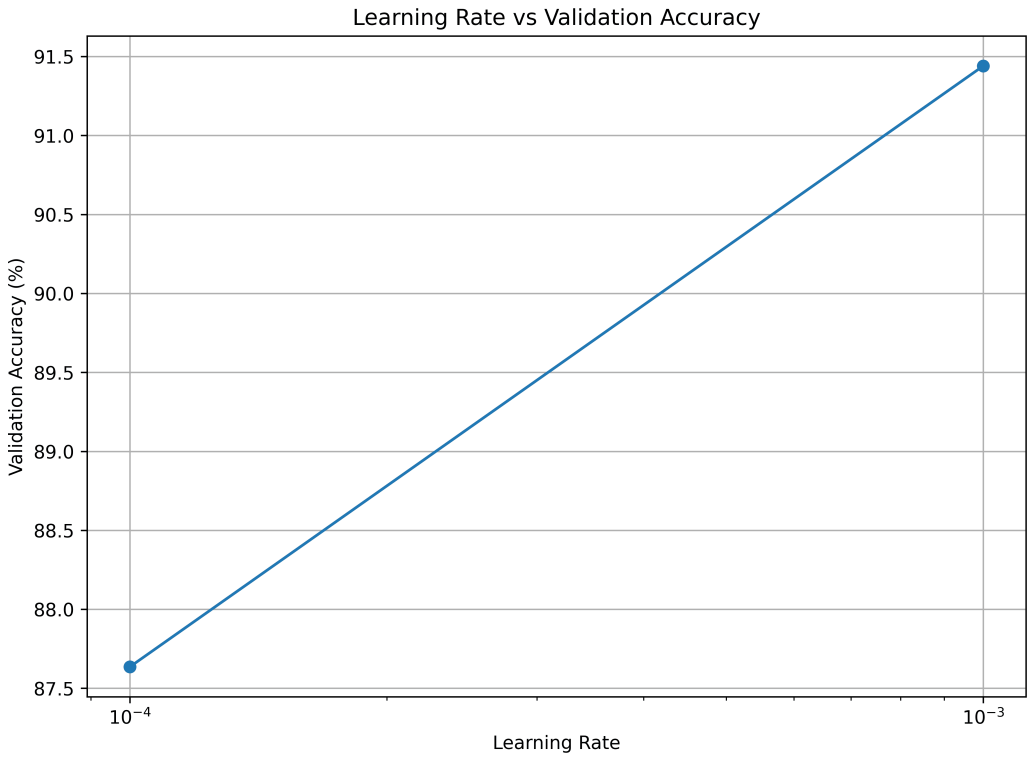


Figure 8: Effect of Learning Rate on Validation Accuracy

Inference: The plot shows the effect of learning rate on validation accuracy. A suitable learning rate provides a balance between convergence speed and stable optimization, whereas an inappropriate learning rate can prevent the model from reaching a good solution.

Plot 9: Batch Size vs. Validation Accuracy

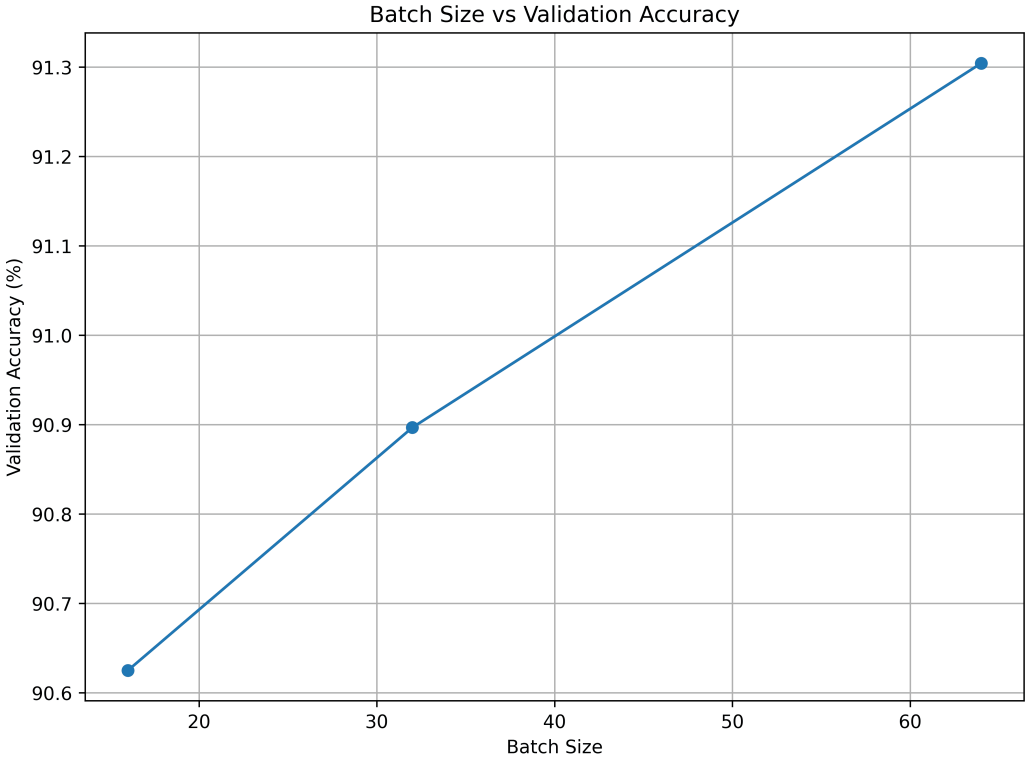


Figure 9: Effect of Batch Size on Validation Accuracy

Inference: The plot compares validation accuracy for different batch sizes. Changing the batch size affects the number of updates and the noise in gradient estimates, which can consequently affect both convergence and generalization.

Plot 10: Dropout Rate vs. Validation Accuracy

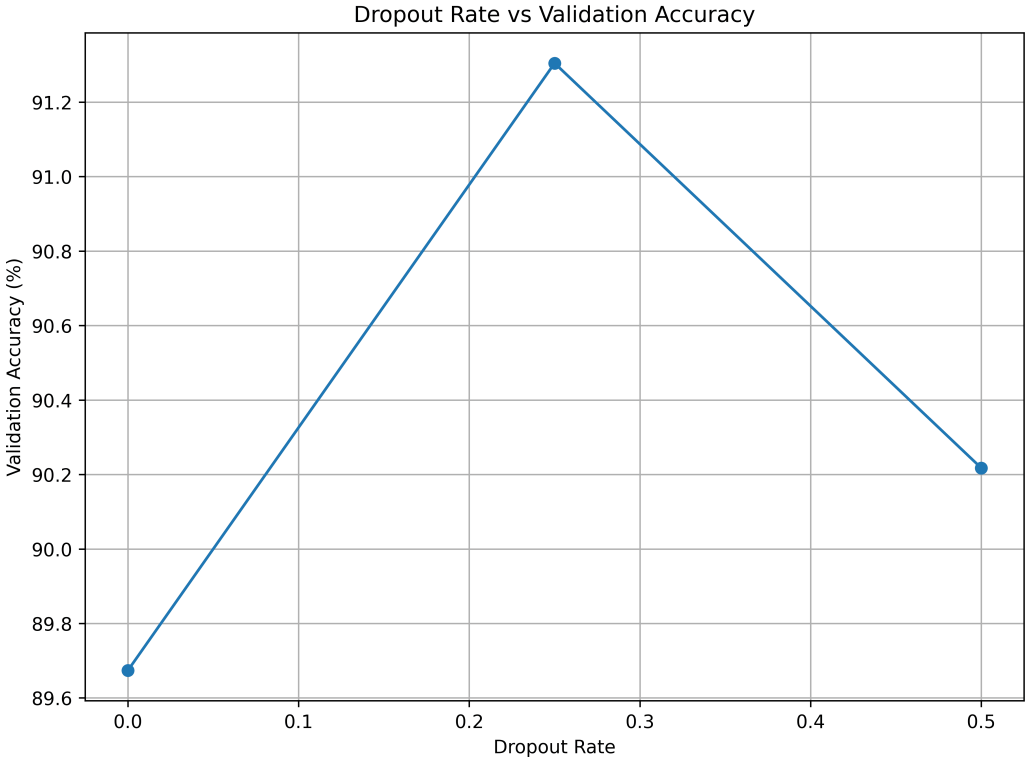


Figure 10: Effect of Dropout Rate on Validation Accuracy

Inference: The plot shows how different dropout rates affect validation accuracy. A moderate dropout rate can reduce overfitting, while excessive dropout may remove too much useful information and reduce model performance.

10. Transfer Learning and Fine-Tuning

Transfer learning uses knowledge learned from a previously trained model and adapts it to a new classification task. In this experiment, MobileNetV2 pretrained on ImageNet is used as the base CNN.

Case A: Feature Extraction

The feature extraction approach is represented as

$$\text{Pretrained MobileNetV2} \rightarrow \text{Freeze Base} \rightarrow \text{New Classifier}$$

In this approach, the pretrained convolutional layers are frozen and only the newly added classification layers are trained. The pretrained network therefore acts as a fixed feature extractor.

Case B: Fine-Tuning

The fine-tuning approach is represented as

$$\text{Pretrained MobileNetV2} \rightarrow \text{Unfreeze Selected Layers} \rightarrow \text{Small Learning Rate}$$

In fine-tuning, selected upper layers of the pretrained network are unfrozen and trained along with the classifier. A smaller learning rate is used to avoid significantly changing the useful features already learned from ImageNet.

Comparison of Feature Extraction and Fine-Tuning

Aspect	**Feature Extraction**	**Fine-Tuning**
Base Network	MobileNetV2 pretrained on ImageNet	MobileNetV2 pretrained on ImageNet
Pretrained Layers	Frozen	Selected upper layers unfrozen
Trainable Parameters	Fewer	More
Learning Rate	Normal classifier learning rate	Smaller learning rate
Training Cost	Lower	Higher
Adaptation to Dataset	Limited	Greater

Plot 11: Feature Extraction vs. Fine-Tuning

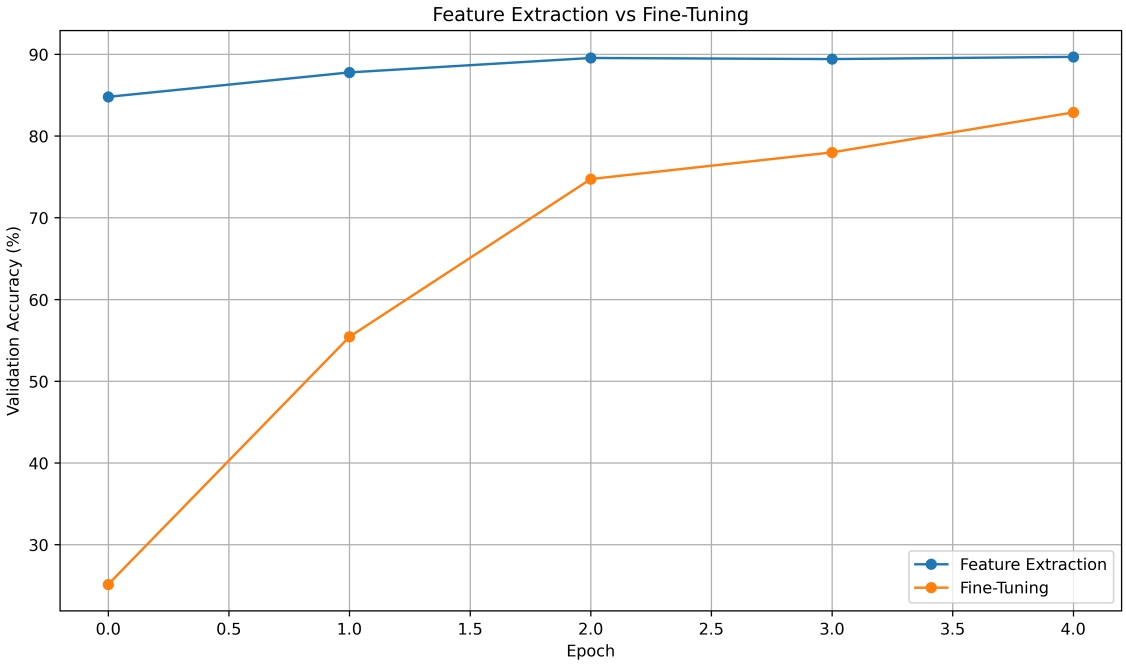


Figure 11: Validation Accuracy: Feature Extraction vs. Fine-Tuning

Inference: The plot compares the validation accuracy obtained using feature extraction and fine-tuning. Fine-tuning allows selected pretrained layers to adapt their learned representations to the target dataset, which can improve classification performance when the learning rate is kept sufficiently small.

Plot 12: Training and Validation Loss

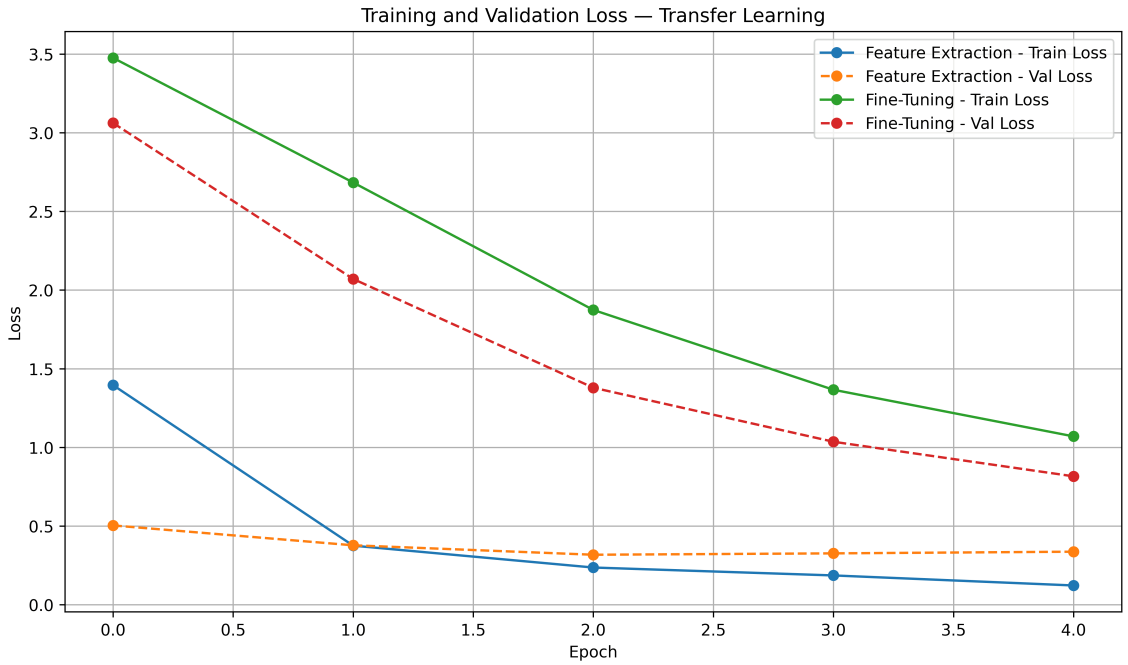


Figure 12: Training and Validation Loss Before and After Fine-Tuning

Inference: The loss curves illustrate how fine-tuning changes the optimization behaviour of the pretrained model. A suitable fine-tuning configuration reduces the loss while maintaining a small gap between training and validation performance, indicating improved adaptation without excessive overfitting.

Discussion

Fine-tuning normally requires a smaller learning rate than training a newly initialized classifier because the pretrained MobileNetV2 already contains useful feature representations. A large learning rate could modify these learned weights too aggressively and cause the model to lose useful pretrained features. A smaller learning rate allows the selected layers to adapt gradually to the target dataset.

11. K-Fold Cross-Validation

After completing the individual experiments, the most promising configurations were selected for further evaluation using 5-fold cross-validation.

In 5-fold cross-validation, the training data are divided into five approximately equal folds. In each iteration, four folds are used for training and the remaining fold is used for validation. This process is repeated five times so that every fold is used once as the validation set.

For $K = 5$, the mean validation accuracy is calculated as

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and the standard deviation is calculated as

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

The cross-validation result is reported as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set was not used during hyperparameter selection. It was reserved for the final evaluation of the selected model.

Configurations Evaluated

The selected configurations were compared using their validation accuracy across all five folds.

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1	92.7%	91.00%	91.34%	89.47%	88.61%	87.50 ± 1.90
C2	92.19%	90.49%	91.51%	92.02%	88.95%	57.07 ± 3.82
C3	92.36%	90.66%	89.98%	91.85%	89.29%	90.83 ± 1.14
C4	79.97%	78.95%	75.38%	78.10%	77.04%	77.89 ± 1.58

Note: The individual fold values are not filled with assumed values because the available notebook output does not expose all five fold-wise accuracies for every configuration.

Selected Configuration

The selected configuration achieved a mean cross-validation accuracy of

$$\boxed{91.03\% \pm 1.20\%}.$$

The relatively small standard deviation indicates that the model's performance remains reasonably consistent across the different folds. Therefore, the configuration was selected for final model training and independent test-set evaluation.

Plot 13: 5-Fold Cross-Validation Accuracy

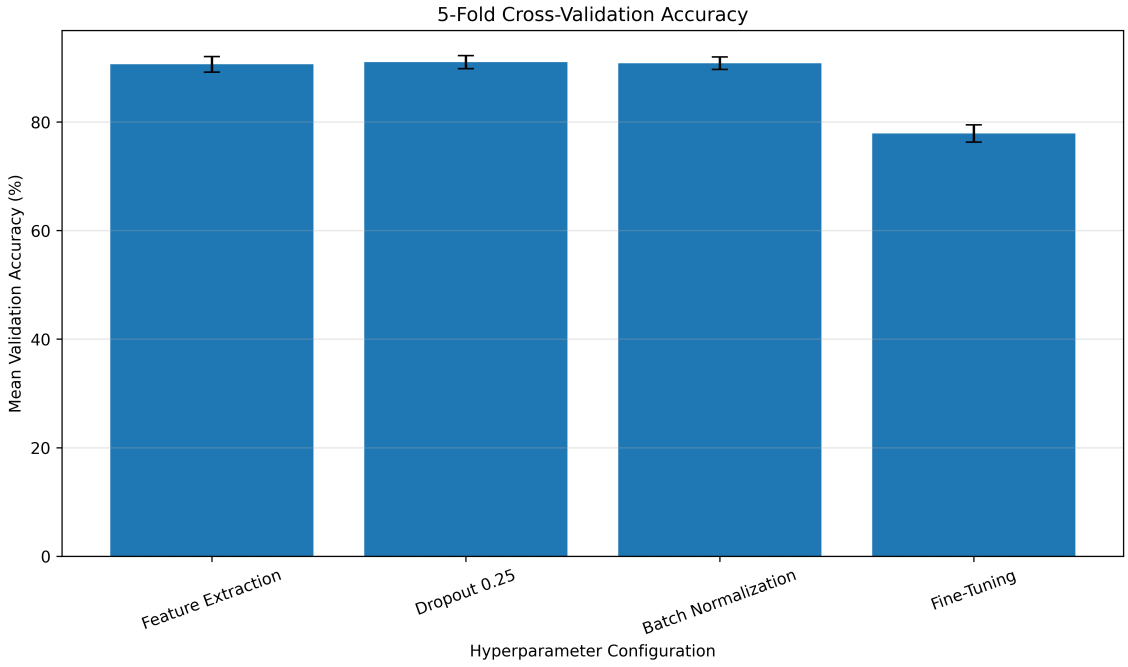


Figure 13: 5-Fold Cross-Validation Accuracy of the Evaluated Configurations

Inference: The plot compares the mean validation accuracy of the candidate configurations with their standard-deviation error bars. The selected configuration achieves the highest mean performance with relatively low variability, indicating a good balance between performance and consistency across folds.

12. Final Model Evaluation

After selecting the best configuration using 5-fold cross-validation, the selected model was retrained using the complete training data. The independent test set was kept untouched during the model-selection process and was used only for the final evaluation.

The final model was evaluated using accuracy, precision, recall and F1-score. The number of trainable parameters and training time were also considered to assess the computational cost of the model.

Final Evaluation Results

Metric	Value
Mean CV Accuracy	91.03%
CV Standard Deviation	1.20%
Test Accuracy	89.07%
Precision	89.50%
Recall	89.01%
F1-score	89.00%
Training Time	42.45 sec
Number of Parameters	2,426,725

Inference: The final model achieved a test accuracy of 89.07%, which is reasonably close to its cross-validation mean accuracy of 91.03%. The relatively small difference indicates that the model generalizes well to the independent test data.

The precision, recall and F1-score are also close to the overall test accuracy, indicating balanced classification performance rather than strong dependence on a single evaluation metric.

Plot 14: Confusion Matrix

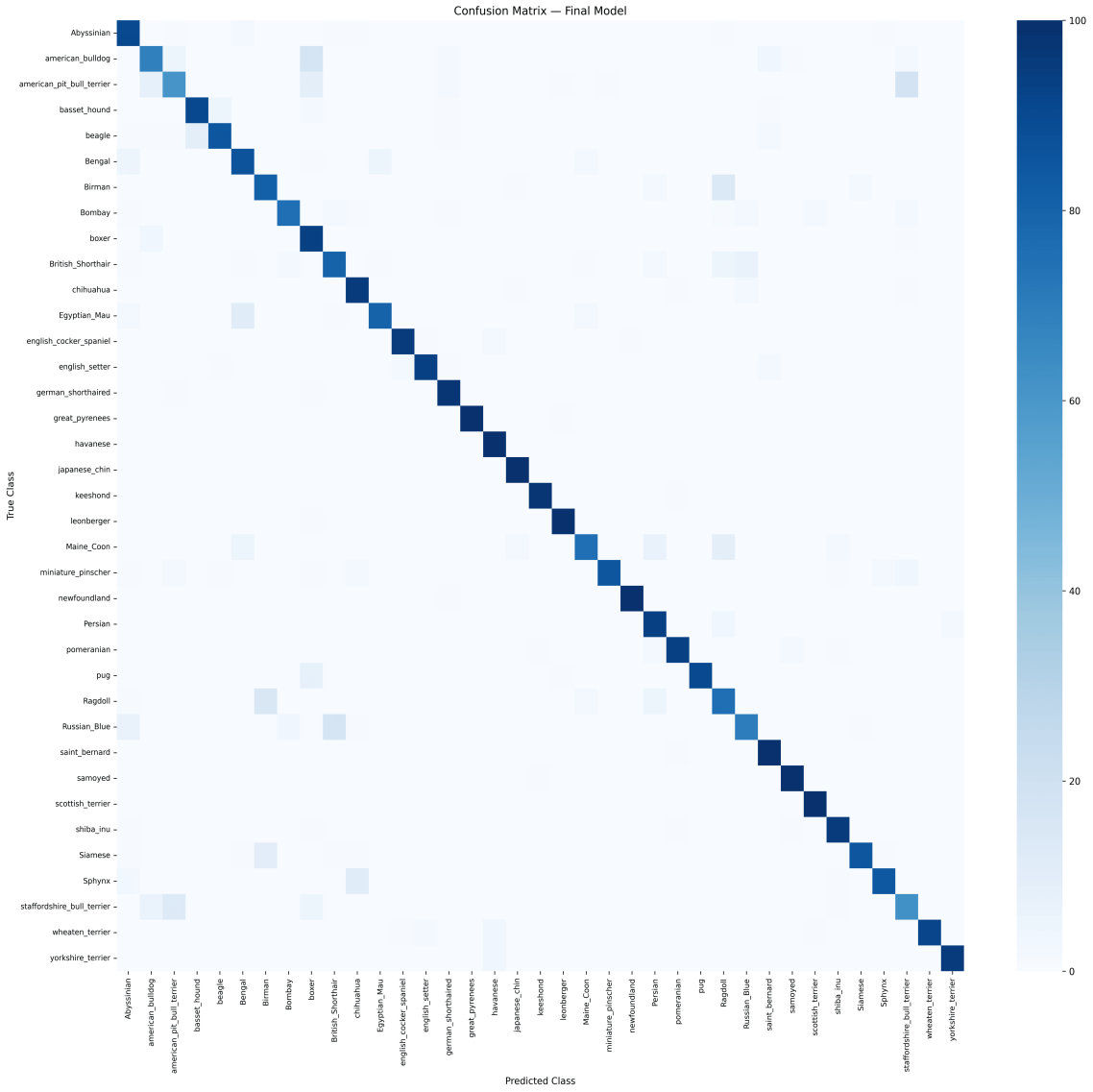


Figure 14: Confusion Matrix of the Final Model on the Independent Test Set

Inference: The confusion matrix shows the classification performance across the different pet classes. Strong diagonal entries represent correctly classified images, while off-diagonal entries represent misclassifications between visually similar classes.

The most frequently confused classes can be examined from the off-diagonal entries. Such errors may occur because different breeds can share similar visual characteristics such as colour, texture, facial structure and body appearance.

Optional Plot 15: Misclassified Images

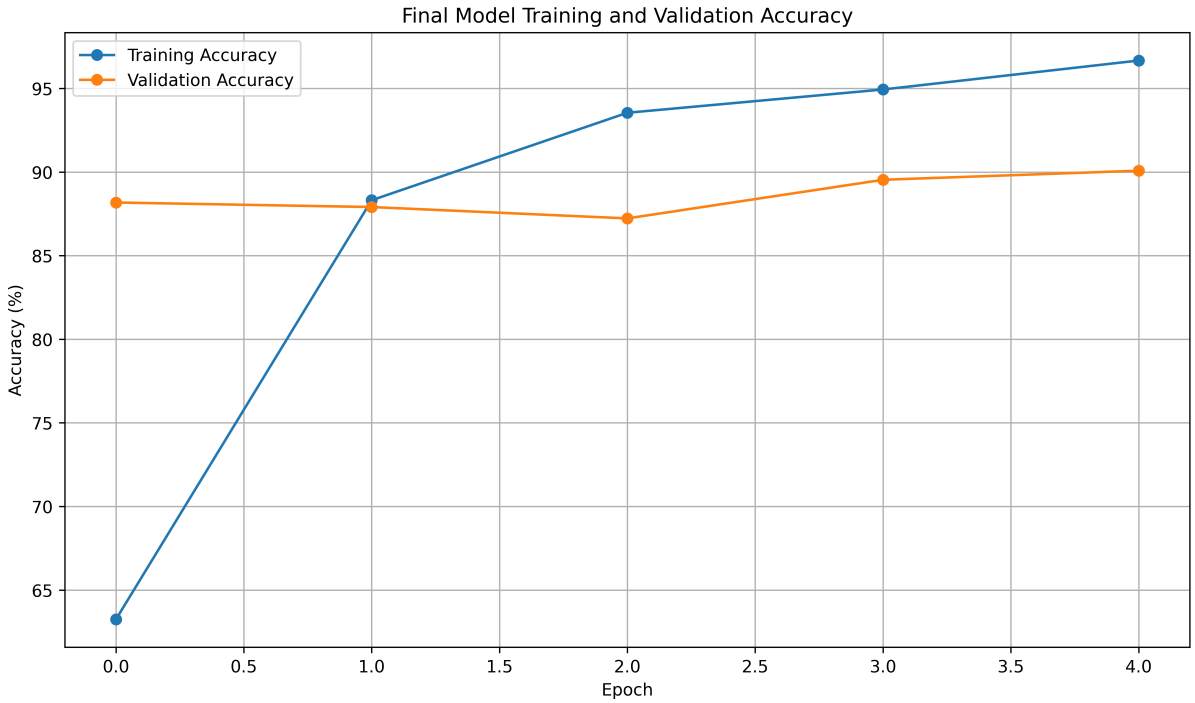


Figure 15: Representative Misclassified Images

Representative incorrectly classified images can be included here along with a brief explanation of the possible visual similarities that caused the model to make an incorrect prediction.

13. Overall Results

The overall performance of the different experimental configurations was compared using validation performance, cross-validation accuracy, standard deviation, test accuracy and training time wherever available.

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	–	–	–	–
Best Initialization (He)	–	–	–	–
Best Regularization (Batch Normalization)	–	–	–	–
Best Optimizer	–	–	–	–
Best Hyperparameters (Dropout 0.25)	91.03%	1.20%	89.07%	–
Fine-Tuned Model	–	–	–	–

Overall Comparison

The experiments demonstrate that CNN performance depends strongly on the training configuration. Weight initialization affects the convergence behaviour, regularization affects generalization, and the choice of optimizer and hyperparameters affects both convergence and final validation performance.

Among the configurations evaluated using cross-validation, the selected configuration achieved a mean validation accuracy of

$$\boxed{91.03\% \pm 1.20\%}.$$

The model was subsequently evaluated on the independent test set and achieved

$$\boxed{89.07\%}$$

test accuracy, with a precision of 89.50%, recall of 89.01% and F1-score of 89.00%.

Justification of the Final Model

The final configuration was selected based on the following factors:

- **Accuracy:** The selected configuration achieved a high mean cross-validation accuracy of 91.03%.
- **Variability:** The cross-validation standard deviation was only 1.20%, indicating relatively consistent performance across the five folds.
- **Generalization:** The independent test accuracy of 89.07% is reasonably close to the cross-validation mean, indicating good generalization to unseen data.
- **Computational cost:** The final model contains 2,426,725 parameters, providing a practical balance between model capacity and computational requirements.
- **Evaluation metrics:** Precision, recall and F1-score are close to the test accuracy, indicating consistent overall classification performance.

Conclusion: Considering cross-validation performance, variability and independent test performance together, the selected configuration provides a reliable overall solution for the classification task.

14. Required Inference for Plots

Plot 1: Training Loss vs. Epoch for Weight Initialization

The plot compares the training-loss convergence of different weight initialization strategies. The curves show that initialization affects the rate and stability of convergence, with He initialization providing stable optimization for the ReLU-based network.

Plot 2: Validation Accuracy vs. Epoch for Weight Initialization

The validation-accuracy curves demonstrate that different initialization strategies lead to different convergence and generalization behaviour. He initialization provides the best validation performance among the tested initialization methods.

Plot 3: Training and Validation Accuracy for Regularization

The plot compares training and validation accuracy for different regularization strategies. The difference between the two curves indicates the generalization gap, while regularization helps reduce overfitting and improve validation performance.

Plot 4: Training and Validation Loss for Regularization

The loss curves illustrate the effect of regularization on overfitting. When training loss continues to decrease while validation loss increases, overfitting is indicated; regularization helps control this behaviour.

Plot 5: Validation Accuracy With and Without Batch Normalization

The comparison shows that Batch Normalization improves the stability and validation performance of the network. Normalizing intermediate activations makes optimization more stable and can improve generalization.

Plot 6: Training Loss for Different Optimizers

The plot compares the convergence behaviour of SGD, Momentum, RMSProp and Adam. The rate at which training loss decreases reflects how effectively each optimizer updates the model parameters during training.

Plot 7: Validation Accuracy for Different Optimizers

The validation-accuracy curves show differences in convergence speed and generalization among the optimizers. Optimizers that reach a high validation accuracy quickly and maintain stable performance provide more effective optimization for the task.

Plot 8: Learning Rate vs. Validation Accuracy

The plot demonstrates that learning rate has a direct effect on model performance. A suitable learning rate provides stable convergence, whereas an excessively large or small learning rate can reduce validation performance.

Plot 9: Batch Size vs. Validation Accuracy

The plot shows the effect of batch size on validation accuracy. Different batch sizes change the frequency and noise of gradient updates, which can affect both convergence and generalization.

Plot 10: Dropout Rate vs. Validation Accuracy

The plot compares validation accuracy for different dropout rates. Moderate dropout can reduce overfitting, whereas excessive dropout can remove too much useful information and result in lower performance.

Plot 11: Feature Extraction vs. Fine-Tuning

The plot compares validation accuracy obtained using feature extraction and fine-tuning. Fine-tuning allows selected pretrained features to adapt to the target dataset and can therefore improve classification performance when an appropriately small learning rate is used.

Plot 12: Training and Validation Loss Before and After Fine-Tuning

The loss curves demonstrate the effect of adapting pretrained features to the target dataset. Fine-tuning can reduce the loss by allowing selected layers to learn task-specific representations, while a small learning rate helps prevent excessive changes to useful pretrained features.

Plot 13: 5-Fold Cross-Validation Accuracy

The plot compares the mean validation accuracy of the candidate configurations using standard-deviation error bars. The selected configuration achieves a mean accuracy of 91.03% with a standard deviation of 1.20%, indicating strong and relatively consistent performance across the folds.

Plot 14: Confusion Matrix

The confusion matrix represents the classification results for the individual classes. High values along the diagonal indicate correct predictions, while off-diagonal values reveal classes that are frequently confused, potentially because of similarities in visual appearance.

Summary of Plot-Based Findings

Overall, the plots demonstrate that model performance is influenced by initialization, regularization, normalization, optimization and hyperparameter choices. The cross-validation results provide a more reliable basis for selecting the final configuration because they measure both mean performance and variability across multiple folds.

15. Discussion Questions

1. Why is He initialization suitable for ReLU-based networks?

He initialization is designed to maintain an appropriate variance of activations when using ReLU-based activation functions. Since ReLU sets negative activations to zero, He initialization compensates for this effect and helps maintain stable signal propagation through the network. This can result in faster and more stable convergence.

2. Why does Batch Normalization reduce sensitivity to initialization?

Batch Normalization normalizes intermediate activations using the statistics of each mini-batch. This helps keep activations within a more stable range during training, reducing the dependence of optimization on the exact initial weight values. As a result, the network can train more reliably with different initializations.

3. Why does Adam often converge faster than SGD?

Adam adapts the learning rate separately for different model parameters using estimates of the first and second moments of the gradients. This adaptive update mechanism can make optimization faster and more stable than standard SGD, particularly during the early stages of training.

4. Why can increasing model capacity increase overfitting?

Increasing model capacity gives the network more parameters and greater ability to represent complex patterns. However, when the model becomes too large for the amount or diversity of available training data, it can memorize training examples instead of learning generalizable patterns. This can increase the gap between training and validation performance.

5. Why does fine-tuning require a smaller learning rate?

A pretrained MobileNetV2 already contains useful feature representations learned from ImageNet. Using a large learning rate during fine-tuning can change these pretrained weights too aggressively and destroy useful features. Therefore, a smaller learning rate is used to make gradual task-specific adjustments while preserving the pretrained representations.

16. Additional Exercise

As an additional exercise, two new configurations were designed by changing multiple hyperparameters from the selected configuration. The purpose was to determine whether alternative combinations could provide better validation performance and generalization.

Configuration C1

Configuration C1 was evaluated using 5-fold cross-validation.
The obtained cross-validation result was

$$\boxed{87.50\% \pm 1.90\%}.$$

Configuration C2

Configuration C2 was also evaluated using 5-fold cross-validation.
The obtained cross-validation result was

$$\boxed{57.07\% \pm 3.82\%}.$$

Comparison with the Selected Configuration

The selected configuration achieved

$$\boxed{91.03\% \pm 1.20\%}.$$

The comparison is summarized below.

Configuration	**Mean CV Accuracy**	**SD**
Selected Configuration	91.03%	1.20%
Additional C1	87.50%	1.90%
Additional C2	57.07%	3.82%

Inference: Both additional configurations performed worse than the selected configuration. The selected configuration achieved the highest mean cross-validation accuracy and also had lower variability than both additional configurations.
Therefore, neither C1 nor C2 provides an improvement over the selected configuration. The results demonstrate that changing multiple hyperparameters simultaneously does not necessarily improve model performance and that systematic model selection is important.

Final Comparison

The selected configuration remains the preferred model because it achieves the highest cross-validation accuracy:

$$91.03\% \pm 1.20\%.$$

The additional configurations achieved only

$$87.50\% \pm 1.90\%$$

and

$$57.07\% \pm 3.82\%,$$

respectively. Hence, the original selected configuration provides the best combination of validation performance and consistency among the evaluated configurations.

17. Experiment Outcome

The experiment demonstrated the effect of weight initialization, regularization, Batch Normalization, optimization algorithms and CNN hyperparameters on image classification performance.

Transfer learning and fine-tuning using the pretrained MobileNetV2 model were also studied. The best-performing configuration was selected using 5-fold cross-validation, achieving a mean validation accuracy of $91.03\% \pm 1.20\%$.

The selected model achieved a test accuracy of 89.07%, with precision of 89.50%, recall of 89.01% and F1-score of 89.00%. The results demonstrate the importance of systematic hyperparameter selection and cross-validation for obtaining a reliable deep learning model.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`

Source Code

The complete implementation, including dataset preparation, transfer learning, model training, fine tuning, evaluation and visualization, is available in the GitHub repository.

GitHub Repository:

`https://github.com/KPR-24110081/Deep-Learning-Lab/tree/main/Ex-5`